

Chapter 13. Logical Architecture and UML Package Diagrams

Objectives

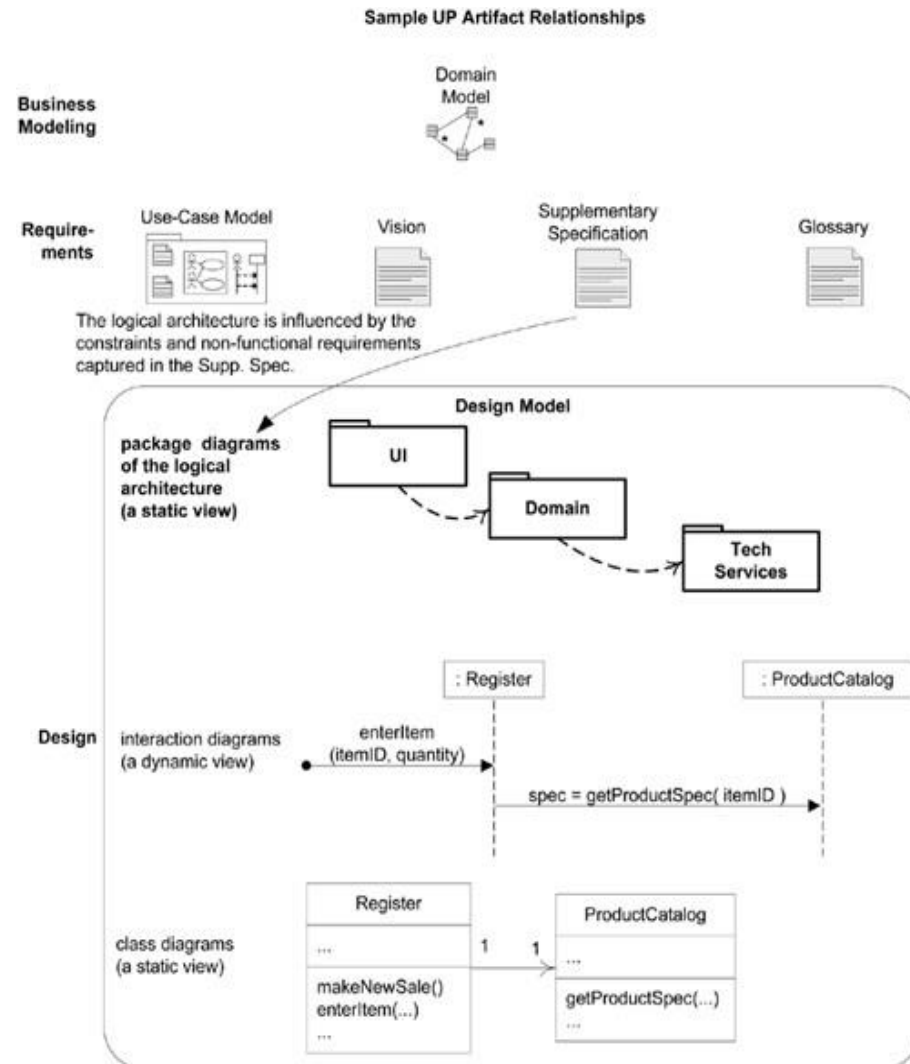
- Introduce a logical architecture using layers.
- Illustrate the logical architecture using UML package diagrams.

Logical Architecture

- This is a very short introduction to the topic of **logical architecture**, a fairly large topic.
- Now we have transitioned from analysis-oriented work to software design.

Sample UP Artifact Relationships

- UP artifacts influence the logical architecture (LA).
 - The **prime input** is the architectural forces captured in the **Supplementary Specification**.
- The logical architecture defines the **packages** within which software classes are defined.
- UML package diagrams**
 - illustrates the LA as part of the **Design Model**.
 - Can be summarized as a view in a **Software Architecture Document**.



What is the Logical Architecture and Layers?

- The **logical architecture** is the large-scale organization of the software classes into packages, subsystems, and layers.
 - No decision about how these elements are deployed across different operating system processes or across physical computers in a network.
 - physical architecture (**UML deployment diagram**)

- A **layer** is a coarse-grained **grouping** of classes, packages, or subsystems that has **cohesive responsibility**
=> 피시컬 아키텍처가 어떻게 개선&발전되는지에 대한 내용은 포함 X

- "higher" layers call upon services of "lower" layers but not vice versa.
=> 비슷한 responsibility를 묶음

- Typical layers in an OO system

상위 • User Interface : 단방향의 dependency를 지키기 위해서.

(상위) • Application Logic and Domain Objects : (on system에서의 레이어 계층) software objects representing domain concepts that fulfill application requirements.

| • Technical Services : general purpose objects and subsystems that provide supporting technical services, such as interfacing with a database or error

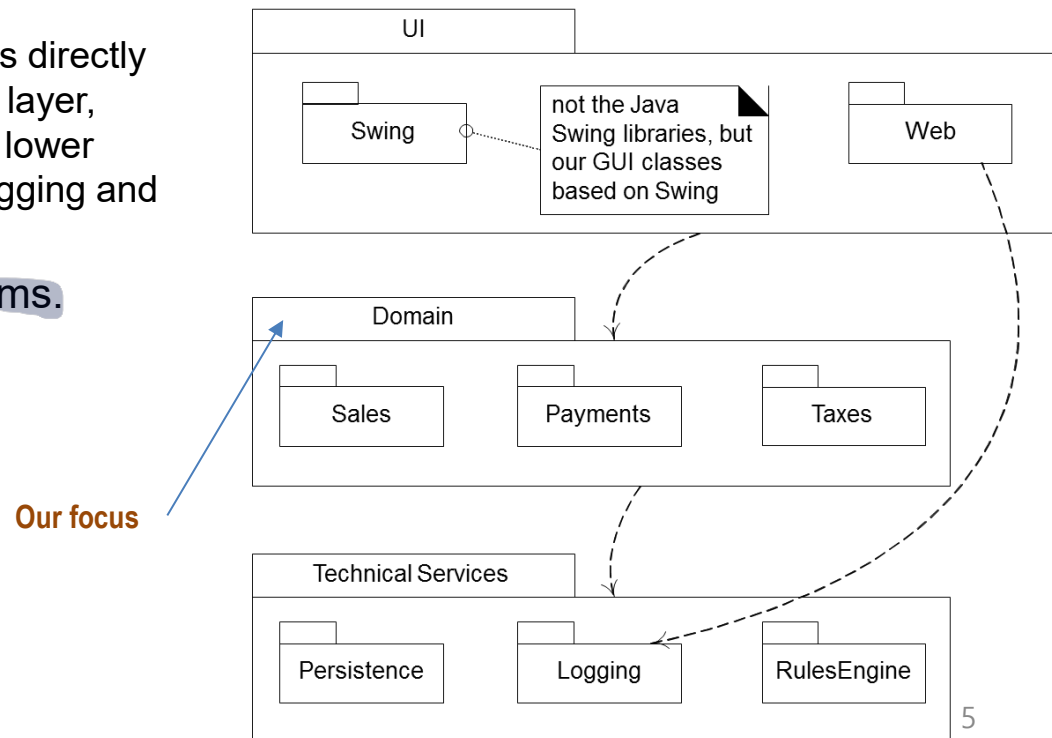
(하위) logging. These services are usually application-independent and reusable

하위가 바뀌어도 across several systems.

상위는 ~~변하지~~ X foundation.

Strict v.s. Relaxed Layered Architecture

- A **strict** layered architecture :
 - A layer only calls upon the services of the **layer directly below** it.
 - Common in network protocol stacks, but not in information systems.
- A **relaxed** layered architecture :
 - a higher layer calls upon several lower layers.
 - Example
 - The UI layer may call upon its directly subordinate application logic layer, and also upon elements of a lower technical service layer, for logging and so forth.
 - **Common in information systems.**



What is Software Architecture?

- Software architecture refers to the fundamental structures of a software system and the discipline of creating such structures and systems. Each structure comprises software elements, relations among them, and properties of both elements and relations.

↳ SW system의 근본구조 &
1 구조를 만드는 윤리,

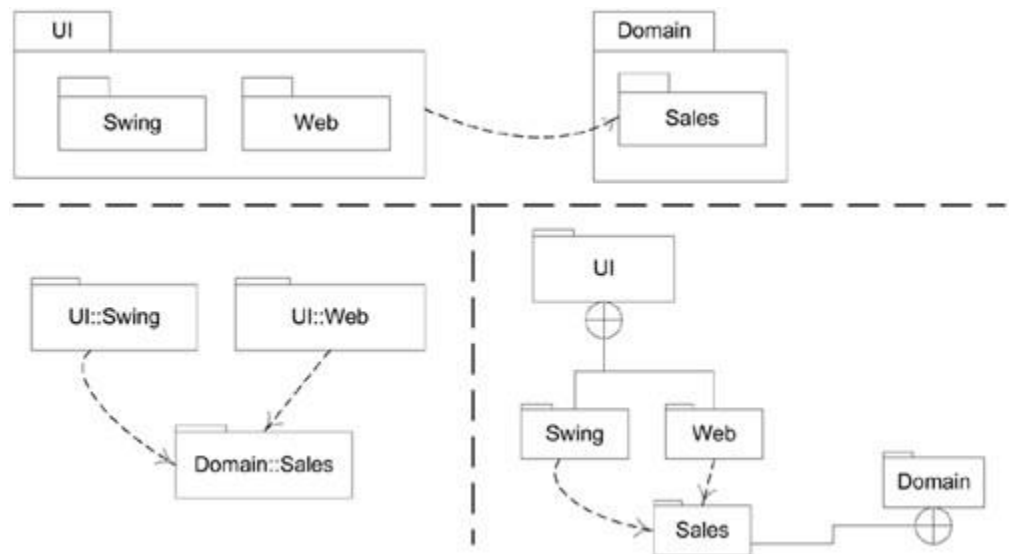
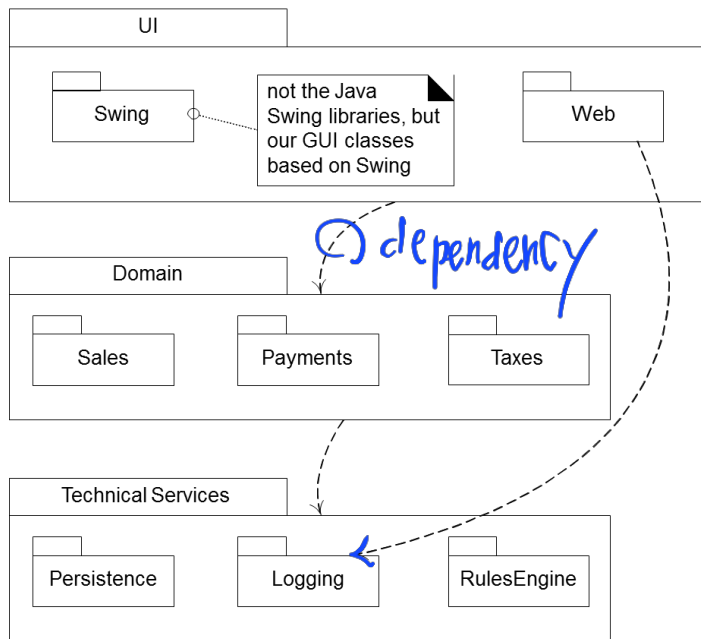
o element, relation, attribute 구성.

Applying UML: Package Diagrams

- **UML package diagrams** are often used to illustrate the logical architecture
 - layers, subsystems, packages (in the Java sense), etc.
 - Example: the UI layer is modeled as a package named UI
- A UML package diagram provides a way to group elements.
 - A UML package can group anything: classes, other packages, use cases, etc.
 - Nesting packages is very common.
 - A UML package is a more general concept than simple a Java package or .NET namespace, though a UML package can represent those.

Package Dependency

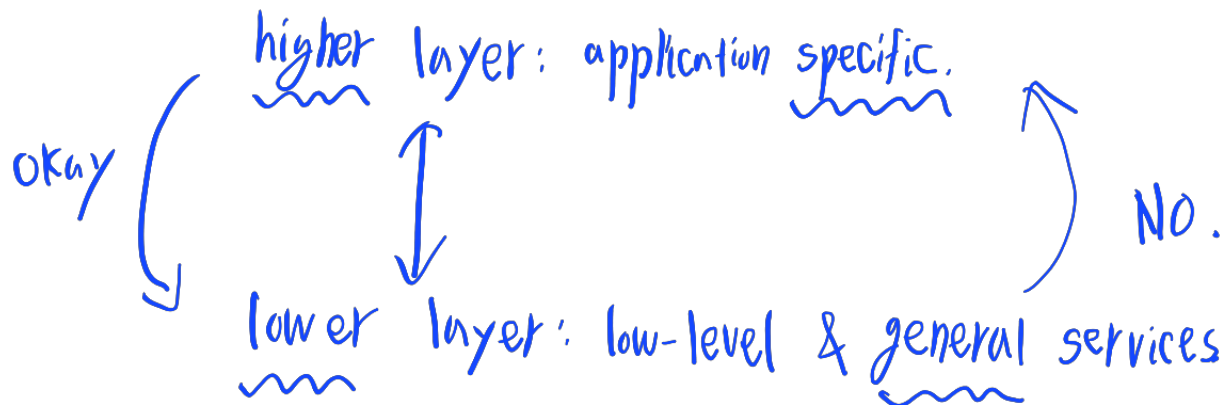
- Dependency (a coupling) between packages
 - Developers can see the large-scale coupling in the system.
 - The UML **dependency** line is used - a dashed arrowed line with the arrow pointing towards the depended - on package.
- Alternate UML approaches to show package nesting:



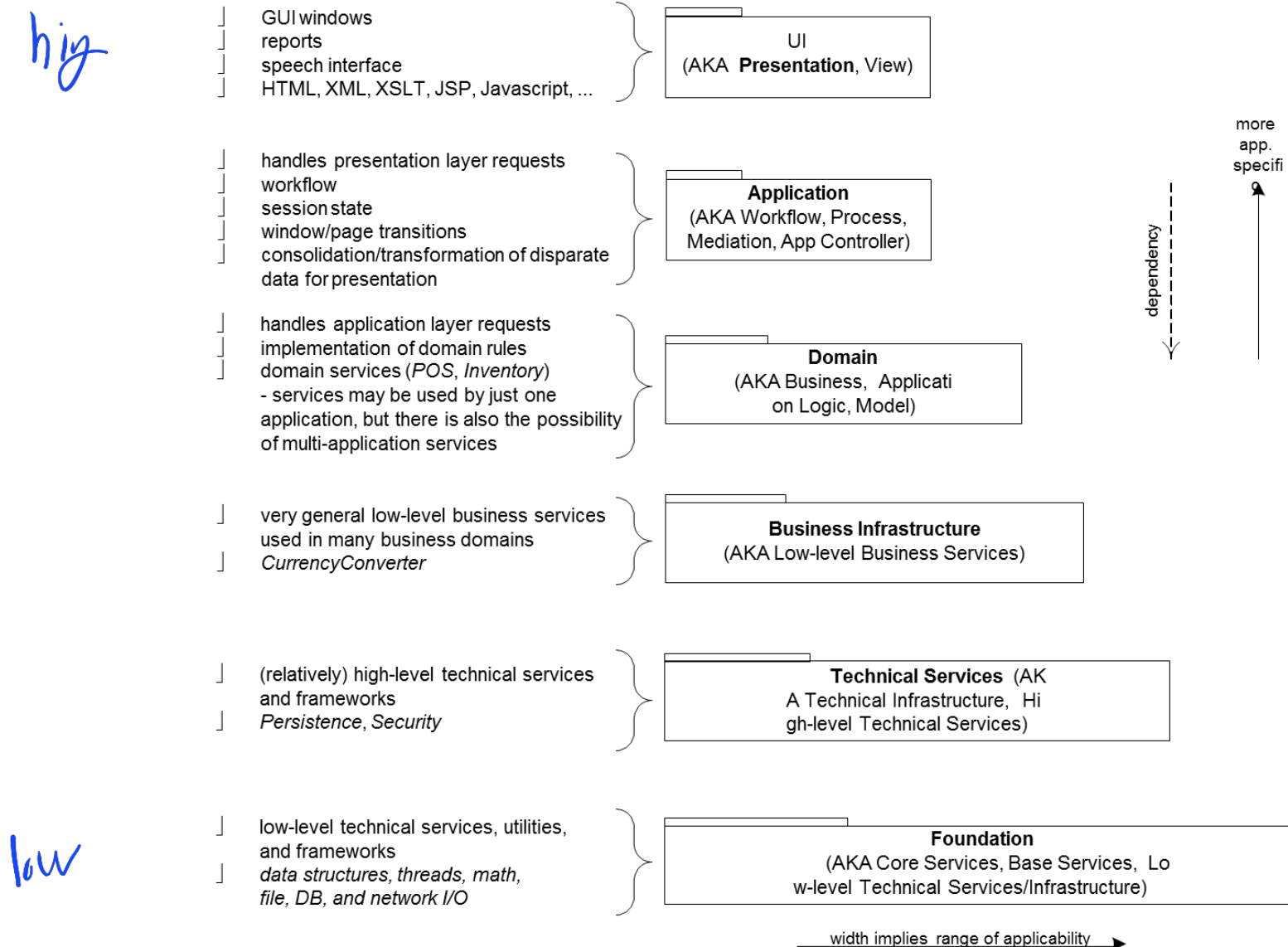
→ 화살표는 종속시키는 방향으로.

Design with Layers

- The essential ideas of using layers
 - Organize the large-scale logical structure of a system into discrete layers of distinct, related responsibilities, with a clean, cohesive separation of concerns such that the **lower layers are low-level and general services**, and the **higher layers are more application specific**.
 - Collaboration and coupling is from higher to lower layers; lower-to-higher layer coupling is avoided.



Example: Common Layers in an Information Systems Logical Architecture

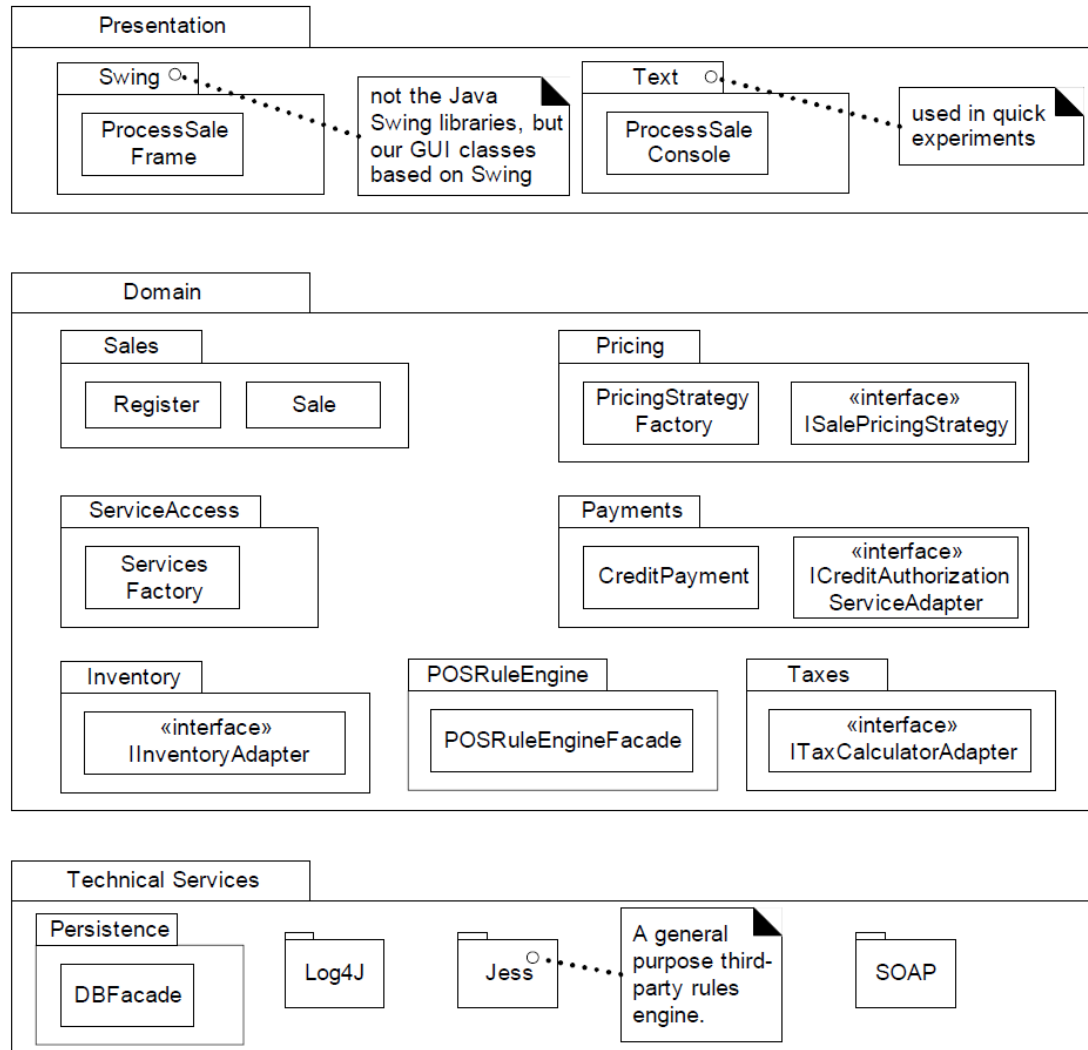


Benefits of Using Layers

이 관심사 분리.

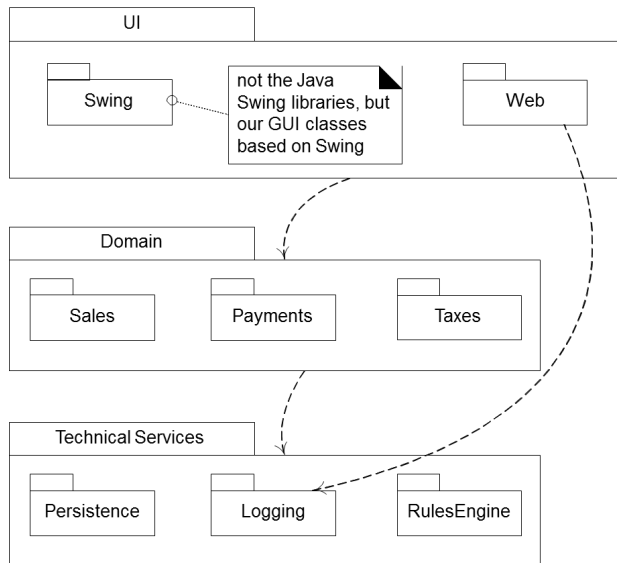
- In general, there is a **separation of concerns**, a **separation of high from low-level services**, and of **application-specific from general** services.
 - This reduces coupling and dependencies, improves cohesion, increases reuse potential, and increases clarity.
- Related complexity is encapsulated and decomposable.
- Some layers can be replaced with new implementations.
 - This is generally not possible for lower-level Technical Service or Foundation layers (e.g., java.util), but may be possible for UI, Application, and Domain layers.
- Lower layers contain reusable functions.
- Some layers (primarily the Domain and Technical Services) can be distributed.
- Development by teams is aided because of the logical segmentation.

Example: Partial LA of NextGen POS



Mapping Code Organization to Layers and UML Packages

- Most popular OO languages (Java, C#, C++, Python, ...) provide support for packages (called namespaces in C# and C++).
 - Example,
 - Use Java to map UML packages to code.
 - Then notice that the layer name is also used as a section of the Java package name.



일관성
통일
시행

```
// --- UI Layer
com.mycompany.nextgen.ui.swing
com.mycompany.nextgen.ui.web

// --- DOMAIN Layer
// packages specific to the NextGen project
com.mycompany.nextgen.domain.sales
com.mycompany.nextgen.domain.payments

// --- TECHNICAL SERVICES Layer
// our home-grown persistence (database) access layer
com.mycompany.service.persistence
// third party
org.apache.log4j
org.apache.soap.rpc

// --- FOUNDATION Layer
// foundation packages that our team creates
com.mycompany.util
```

Domain Layers and Domain Objects

- A typical software system has UI logic and application logic.
Here is a key question :

“How do we design the application logic with objects?”

- The answer is
 - To create software objects with names and information similar to the real-world domain, and then assign application logic responsibilities to them.
 - e.g. real-world POS: sales and payments. In software, *Sale* and *Payment* class, and give them application logic responsibilities.
 - This kind of software object is called a domain object.
 - It represents a thing in the problem domain space, and has related application or business logic,
 - e.g. *Sale* object is able to calculate its total.
- ✓ Domain layer : contains domain objects to handle application logic

Domain Model: 도메인의 실제 개념을 시각화한 것, ≠ Domain layer: Design model 내부의 도메인 클래스를 담당하는 실제 SW Class들의 레이어.

Domain Layer and Domain Model

A Payment in the Domain Model is a concept, but a Payment in the Design Model is a software class. They are not the same thing, but the former *inspired* the naming and definition of the latter.

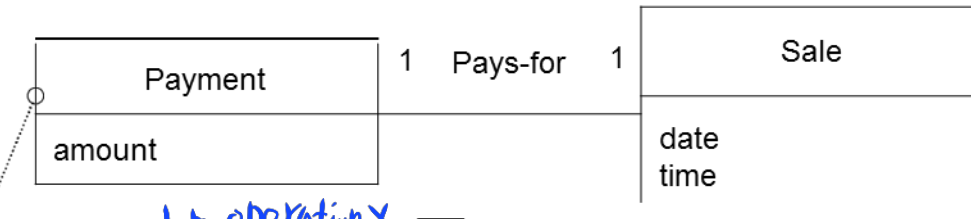
This reduces the representational gap.

This is one of the big ideas in object technology.

* Domain model이
Domain layer의 클래스에
영감을 준다.

UP Domain Model

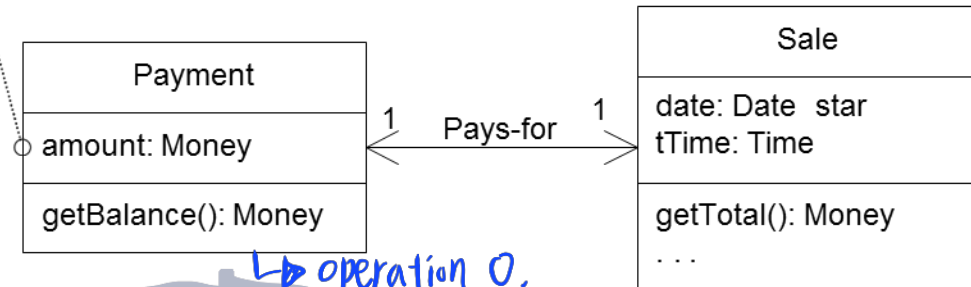
Stakeholder's view of the noteworthy concepts in the domain.



↳ operation X

inspires
objects
and
names in

↑ ↓
이건 유사해야
하니까 & 날짜
같은
유리,



↳ operation O.

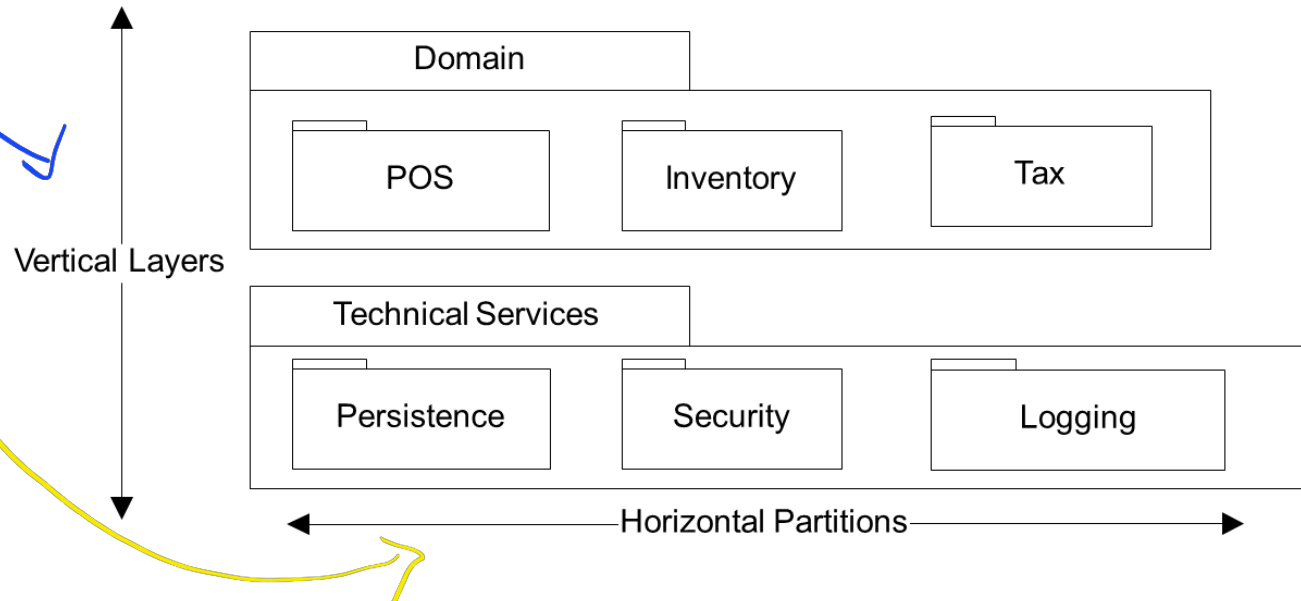
Domain layer of the architecture in the UP Design Model

The object-oriented developer has taken inspiration from the real world domain in creating software classes.

Therefore, the representational gap between how stakeholders conceive the domain, and its representation in software, has been lowered.

Tiers, Layer and Partitions

- **Tier**: widely used to mean a physical processing node (or cluster of nodes), such as the "client tier" (the client computer). 물리적 처리 노드
- **Layers**: represent the vertical slices of the system.
- **Partitions**: represent a horizontal division of relatively parallel subsystems of a layer.
 - For example, the Technical Services layer may be divided into partitions such as Security and Reporting



The Model-View Separation Principle

- **Model-View Separation Principle**

1. **Do not connect** or couple **non-UI objects** directly to **UI objects**.

- For example, don't let a *Sale* software object (a non-UI "domain" object) have a reference to a Java Swing *JFrame* window object. Why? Because the windows are related to a particular application, while (ideally) the non-windowing objects may be reused in new applications or attached to a new interface.

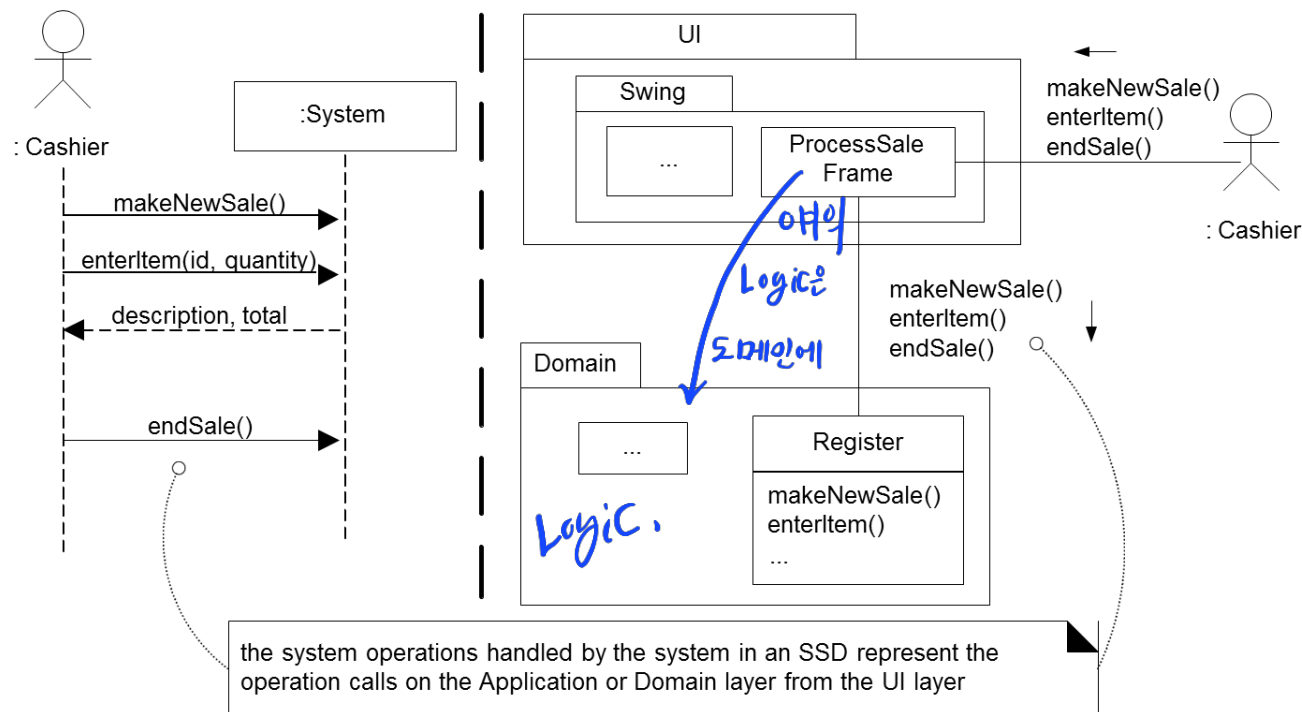
2. **Do not** put **application logic** (such as a tax calculation) **in the UI object methods**.

- UI objects should only initialize UI elements, receive UI events (such as a mouse click on a button), and delegate requests for application logic on to non-UI objects (such as domain objects).

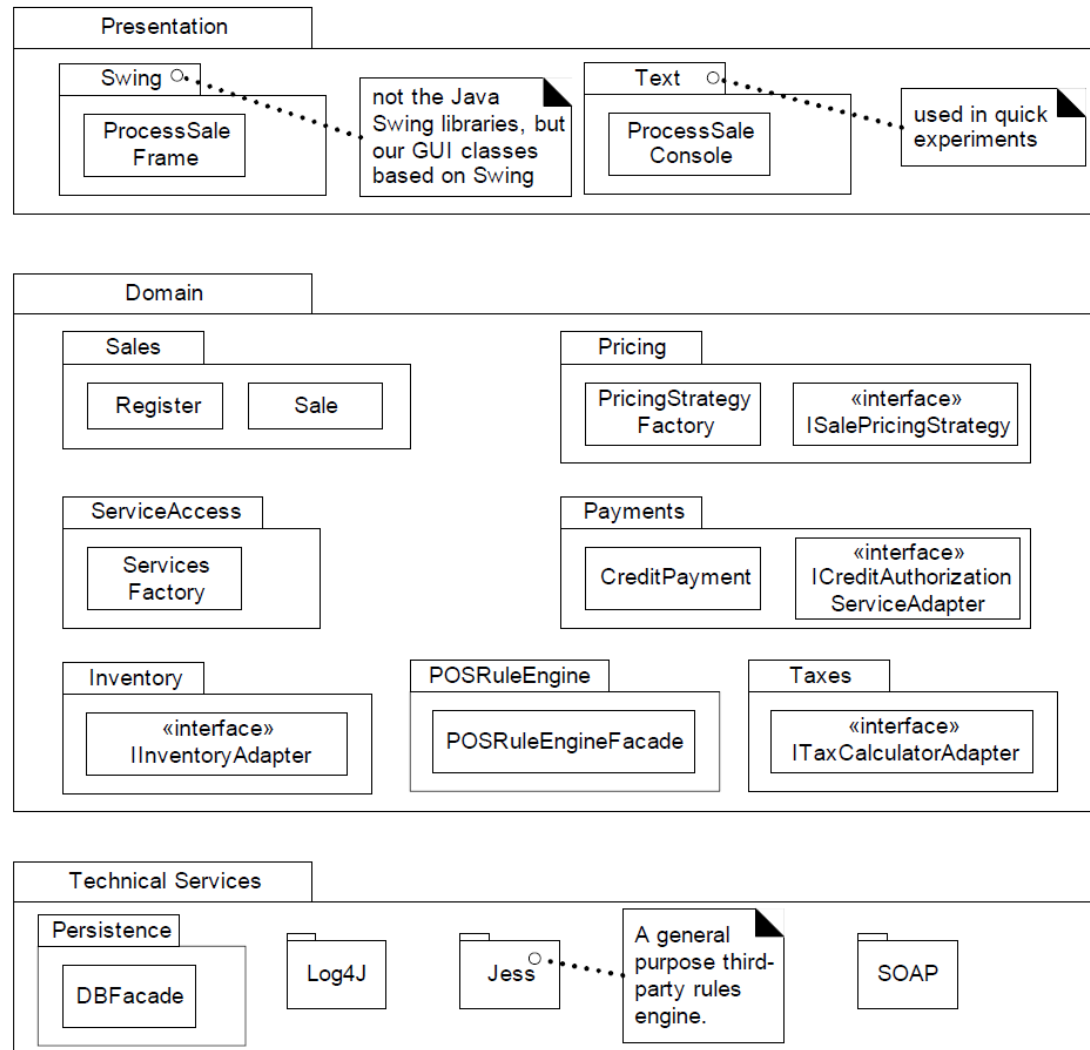
- View → User Interface Layer
- Model → Domain Layer

Connections Between SSDs, System Operations, and Layers

- In a well-designed layered architecture,
 - The **UI layer** objects will **forward** or delegate the **requests** from the UI layer (system operations) onto the **domain layer** for handling.
 - The messages sent from the **UI layer to the domain layer** will be the **messages illustrated on the SSDs** (e.g., *enterItem*).



Example: NextGen POS Logical Architecture and Package Diagram



Partial Package Dependency

